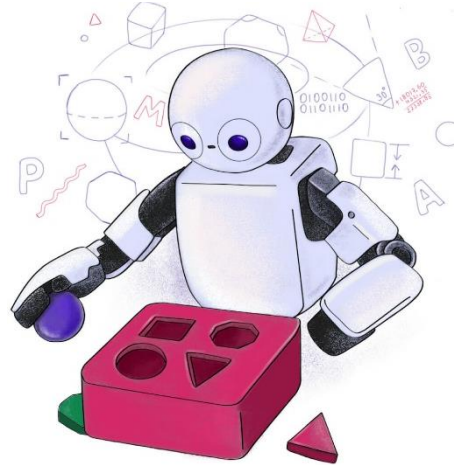
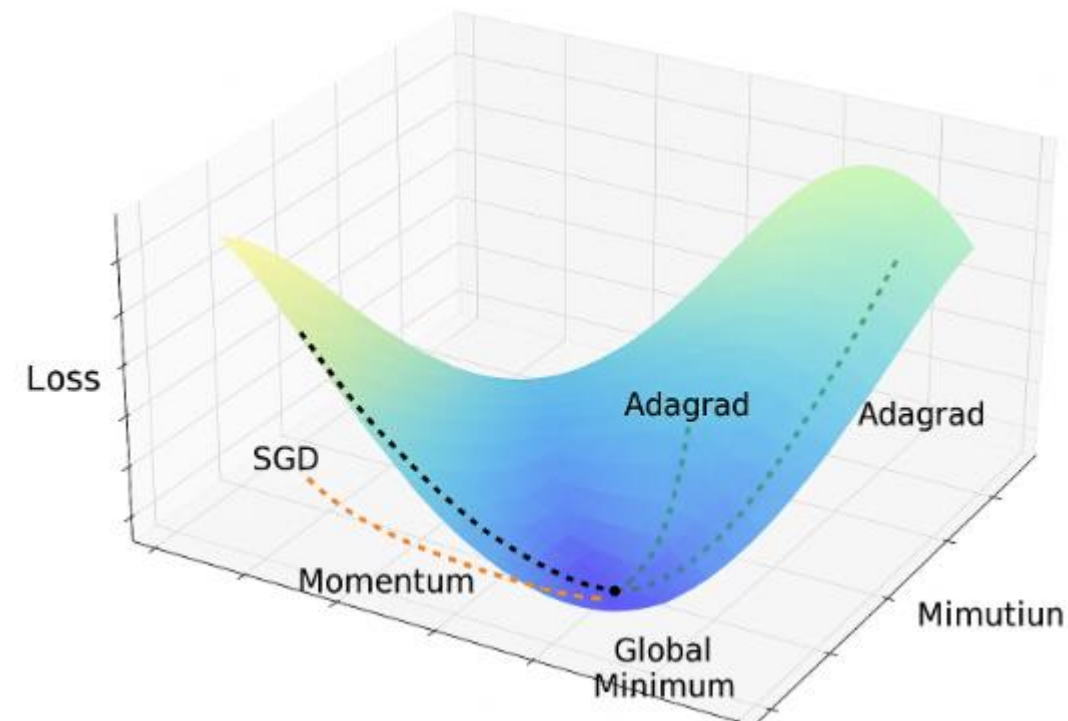


TP558 - Tópicos avançados em Machine Learning: *Gradient Centralization*



Introdução



- O sucesso do aprendizado profundo (Deep Learning) deve-se a vários fatores: grandes volumes de dados, arquiteturas de rede e, principalmente, **técnicas de otimização eficientes**, que permitem treinar redes profundas sem que o processo se torne instável ou lento.
- Métodos como **SGD**, **Momentum**, **Adagrad** e **Adam** têm sido fundamentais para que os modelos converjam e alcancem boa generalização.

Introdução

Problema principal

- Existem técnicas que tentam estabilizar o treinamento:
 - **Batch Normalization (BN)**: normaliza as ativações da rede.
 - **Weight Standardization (WS)**: normaliza os pesos das camadas.
- No entanto, essas técnicas atuam **sobre ativações ou pesos**, e não diretamente sobre os **gradientes**, que são o componente central no processo de aprendizado.

Introdução

Problema principal

- Apesar desses avanços, o treinamento de redes neurais profundas ainda enfrenta desafios como:
 - **Explosão ou desaparecimento dos gradientes** durante o treinamento;
 - **Superfícies de perda irregulares** (difíceis de otimizar);
 - **Desempenho instável** quando se utilizam lotes pequenos de dados.

Introdução

Propuesta dos autores

- Nesse contexto, **o Gradient Centralization (GC)** surge como uma **nova técnica de otimização** que atua diretamente sobre os **gradientes**.
- A ideia central é simples, mas poderosa:

“Centralizar os gradientes para que tenham média zero antes de atualizar os pesos.”

Introdução

Propuesta dos autores

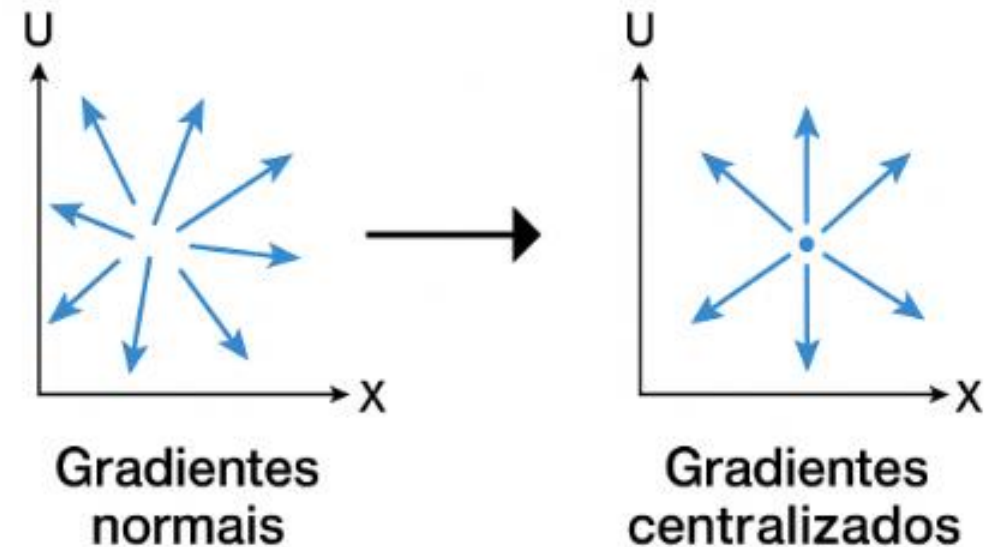
“Centralizar os gradientes para que tenham média zero antes de atualizar os pesos.”

- Isso permite:
 - Suavizar o panorama de otimização (a função de perda torna-se mais estável);
 - Acelerar a convergência;
 - Melhorar a capacidade de generalização do modelo;
- atuando diretamente sobre os **gradientes**, que são o componente central no processo de aprendizado.

Introdução

Objetivo general

- O objetivo do Gradient Centralization (GC) é **regularizar o espaço dos pesos e das características de saída**, tornando o aprendizado mais estável, rápido e com melhor capacidade de generalização. Em resumo:
- O GC busca “**centralizar**” o processo de aprendizado em uma região mais suave e controlada do espaço de parâmetros, melhorando o treinamento sem precisar alterar a arquitetura da rede nem adicionar novos hiperparâmetros.



Fundamentação teórica

Gradient Centralization (GC)

- É uma técnica de otimização que **modifica diretamente os gradientes** durante o treinamento de uma rede neural.

- Seu princípio é simples:

Antes de atualizar os pesos, remove-se a média de cada vetor de gradiente, de modo que ele tenha média zero.

- Isso gera um gradiente mais equilibrado, que melhora a estabilidade e a capacidade de generalização do modelo.

Fundamentação teórica

- Em cada etapa do treinamento, o otimizador (por exemplo, SGD ou Adam) calcula o **gradiente do erro** em relação aos pesos.
- Esse gradiente indica quanto e em que direção cada peso deve ser ajustado.
- O Gradient Centralization consiste em **remover a média do gradiente** antes de utilizá-lo, para mantê-lo centralizado:

$$g = [g_1, g_2, g_3, \dots, g_M] \Rightarrow \hat{g} = g - \text{media}(g)$$

- Dessa forma, a atualização dos pesos torna-se mais equilibrada e estável.

Fundamentação teórica

- Se $\nabla_w L$ representa o gradiente da perda L em relação ao vetor de pesos w , então o operador de *Gradient Centralization* é definido como:

$$\Phi_{GC}(\nabla_w L) = \nabla_w L - \mu_{\nabla_w L}$$

onde:

$$\mu_{\nabla_w L} = \frac{1}{M} \sum_{j=1}^M \nabla_{w_j} L$$

- O resultado é um **novo gradiente centralizado**, com **média zero em cada vetor de peso**.

Fundamentação teórica

Exemplo simples

- Suponha que, em uma camada, você tenha **3 pesos** e o gradiente calculado em uma iteração seja:

$$\begin{aligned}g &= [2, 4, 6] \\ \text{media}(g) &= (2 + 4 + 6)/3 = 4 \\ \hat{g} &= g - 4 = [-2, 0, 2]\end{aligned}$$

Resultado:

- O novo gradiente $\hat{g}$ tem média zero:

$$(-2 + 0 + 2)/3 = 0$$

- As atualizações ficam **mais equilibradas**:
 - um peso aumenta (+2),
 - outro diminui (-2),
 - E outro permanece igual (0).

Fundamentação teórica

Formulação matricial

- Para uma camada totalmente conectada ou convolucional, a formulação pode ser escrita de forma compacta:

$$\Phi_{GC}(\nabla_W L) = P \nabla_W L, \quad P = I - ee^T$$

onde:

- I : matriz identidade,
- $e = \frac{1}{\sqrt{M}} [1, 1, \dots, 1]^T$,
- P : **matriz de projeção** que remove o componente médio do gradiente.

Isso significa que o GC atua como uma projeção linear do gradiente sobre um hiperplano em que a soma dos componentes do gradiente é zero.

Fundamentação teórica

O que acontece se o gradiente for uma matriz?

- Nas redes neurais, os gradientes dos pesos geralmente são organizados em forma de **matriz**, onde cada **linha** corresponde a uma **entrada** e cada **coluna** a um neurônio de **saída**.
- Suponha uma camada com 2 entradas e 3 neurônios de saída. O gradiente calculado para essa camada é:

$$\nabla_W L = \begin{bmatrix} 1 & 3 & 5 \\ 2 & 4 & 6 \end{bmatrix}$$

- Cada coluna representa os gradientes dos pesos que chegam a um neurônio

Fundamentação teórica

- **Passo 1.** Calculamos a média de cada coluna:

Columna	Promedio
1	$(1+2)/2 = 1.5$
2	$(3+4)/2 = 3.5$
3	$(5+6)/2 = 5.5$

- **Passo 2.** Subtraímos essa média de cada elemento da coluna:

$$\Phi_{GC}(\nabla_W L) = \begin{bmatrix} 1 - 1.5 & 3 - 3.5 & 5 - 5.5 \\ 2 - 1.5 & 4 - 3.5 & 6 - 5.5 \end{bmatrix} = \begin{bmatrix} -0.5 & -0.5 & -0.5 \\ +0.5 & +0.5 & +0.5 \end{bmatrix}$$

- Agora, **cada coluna tem média zero.**
- É exatamente isso que o **GC faz na prática:**
centraliza cada vetor de gradiente coluna por coluna, equilibrando as atualizações para cada neurônio.

Fundamentação teórica

Interpretação geométrica

- Do ponto de vista geométrico, o gradiente centralizado

$$\Phi_{GC}(\nabla_W L)$$

pode ser interpretado como uma **projeção do gradiente original sobre um subespaço** do espaço de parâmetros.

- A atualização dos pesos ocorre dentro desse subespaço, o que **impõe uma restrição de equilíbrio** nas variações dos parâmetros.

Fundamentação teórica

Interpretação geométrica

- Em termos matemáticos, isso equivale a resolver um **problema de otimização com restrição linear**:

$$\min_W L(W), \text{ sujeito a } e^T (W - W_0) = 0$$

onde:

- e é um vetor unitário cujos elementos são 1,
- W_0 representa o valor inicial dos pesos.

Fundamentação teórica

Interpretação geométrica

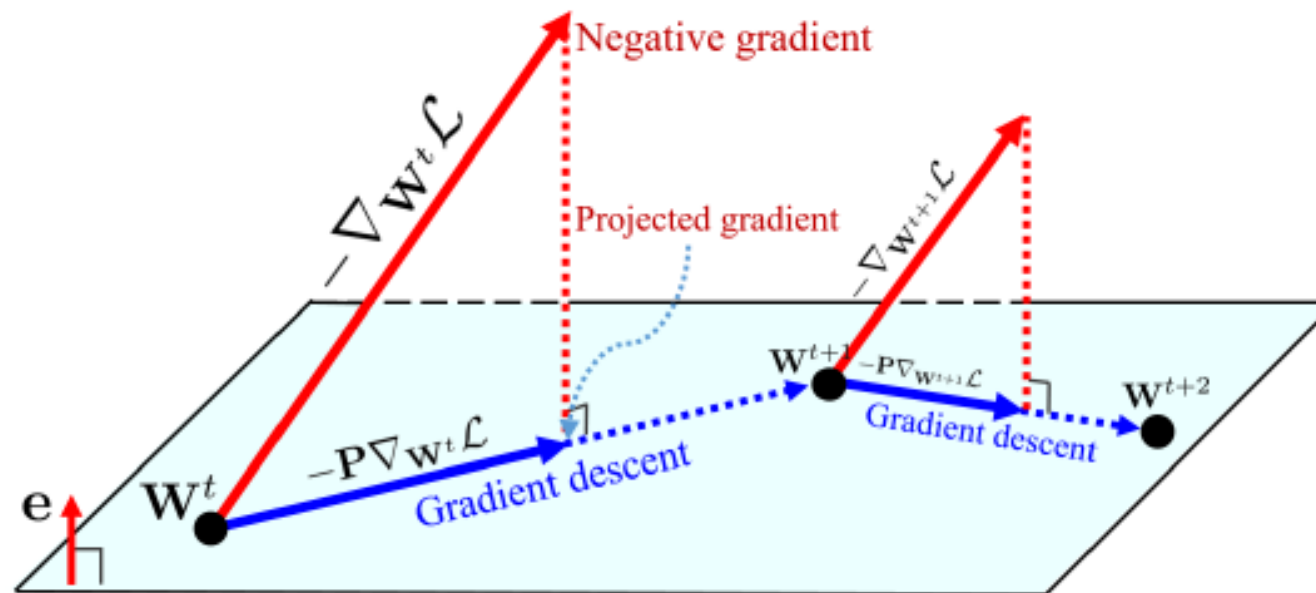


Fig. 2. The geometrical interpretation of GC. The gradient is projected on a hyperplane $e^T(w - w^t) = 0$, where the projected gradient is used to update the weight.

Fundamentação teórica

Propriedades teóricas

a) Regularização implícita

Ao eliminar o componente médio dos gradientes:

- **Reduz-se a redundância** nas atualizações dos pesos.
- **Limita-se o espaço de soluções possíveis**, atuando como uma forma de **regularização natural**.
- Isso **evita o overfitting** sem introduzir novos hiperparâmetros.

Fundamentação teórica

Propriedades teóricas

b) Suavização da paisagem de otimização

- Os autores demonstram que o **GC melhora a Lipschitzness** da função de perda $L(W)$ e de seu gradiente:

$$\begin{aligned} \|\Phi_{GC}(\nabla_W L)\|_2 &\leq \|\nabla_W L\|_2 \\ \|\nabla_W \Phi_{GC}(\nabla_W L)\|_2 &\leq \|\nabla_W^2 L\|_2 \end{aligned}$$

- Isso implica que as **variações da perda em relação aos pesos tornam-se mais suaves**, tornando o treinamento mais **estável e eficiente**.

Fundamentação teórica

c) Estabilização dos gradientes

- A centralização reduz os valores extremos do gradiente (semelhante ao efeito do gradient clipping), evitando:
- Grandes oscilações durante o treinamento.
- Explosão ou desaparecimento dos gradientes.

Fundamentação teórica

Integração em otimizadores existentes

- O GC não substitui os algoritmos de otimização, ele **se integra facilmente** a eles.

- Por exemplo, no **SGD com momentum**, a única modificação é substituir:

$$g_t = \nabla_W L(W_t)$$

por sua versão centralizada:

$$\hat{g}_t = \Phi_{GC}(g_t)$$

- Depois, a atualização continua igual:

$$W_{t+1} = W_t - \alpha \hat{g}_t$$

- O mesmo se aplica a **Adam, Adagrad** ou **qualquer otimizador baseado em gradientes**.

Arquitetura e funcionamento

O GC se posiciona entre a etapa de retropropagação do gradiente e a atualização dos pesos, modificando o fluxo padrão do processo de otimização da seguinte forma:

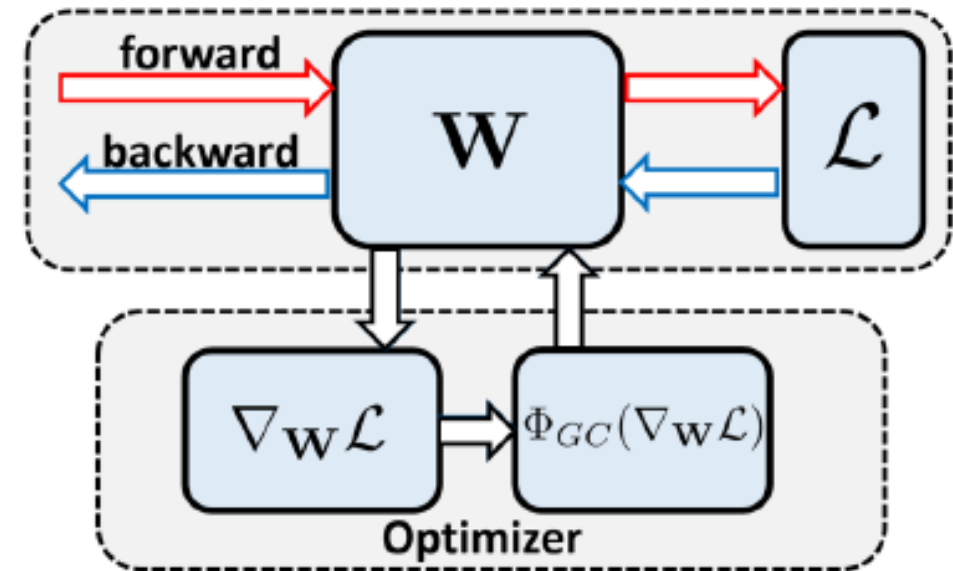
- **Cálculo da perda**

A rede realiza a **propagação direta (forward pass)** e calcula a função de perda $L(W)$ a partir das previsões e dos valores reais.

- **Cálculo do gradiente**

Durante a **retropropagação**, obtém-se o **gradiente da perda em relação aos pesos**:

$$\nabla_W L.$$



Arquitetura e funcionamento

- **Aplicação do Gradient Centralization**

Antes que o otimizador atualize os pesos, o **GC centraliza os gradientes** subtraindo sua média:

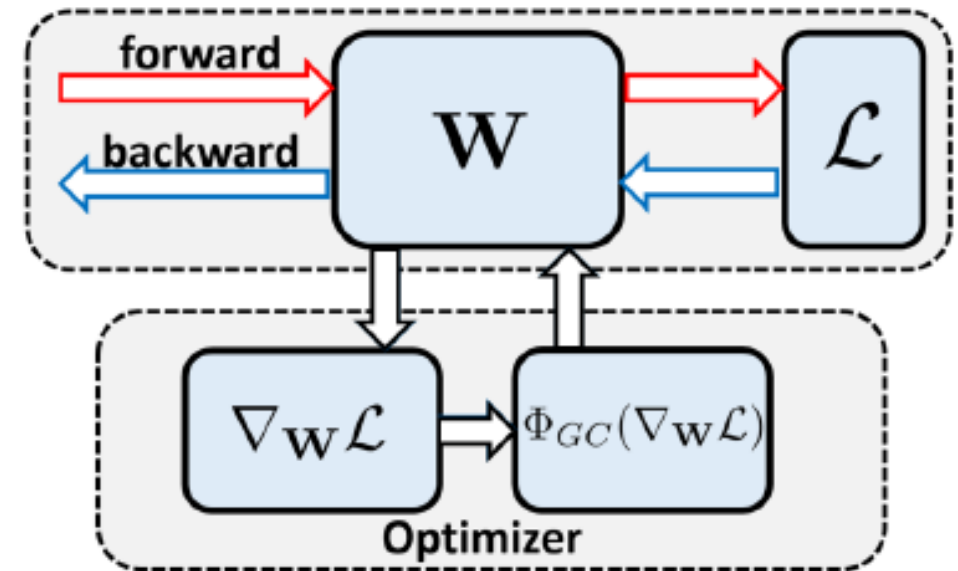
$$\Phi_{GC}(\nabla_W L) = \nabla_W L - \text{mean}(\nabla_W L)$$

Essa operação **equilibra as atualizações dos parâmetros e suaviza o comportamento do gradiente**.

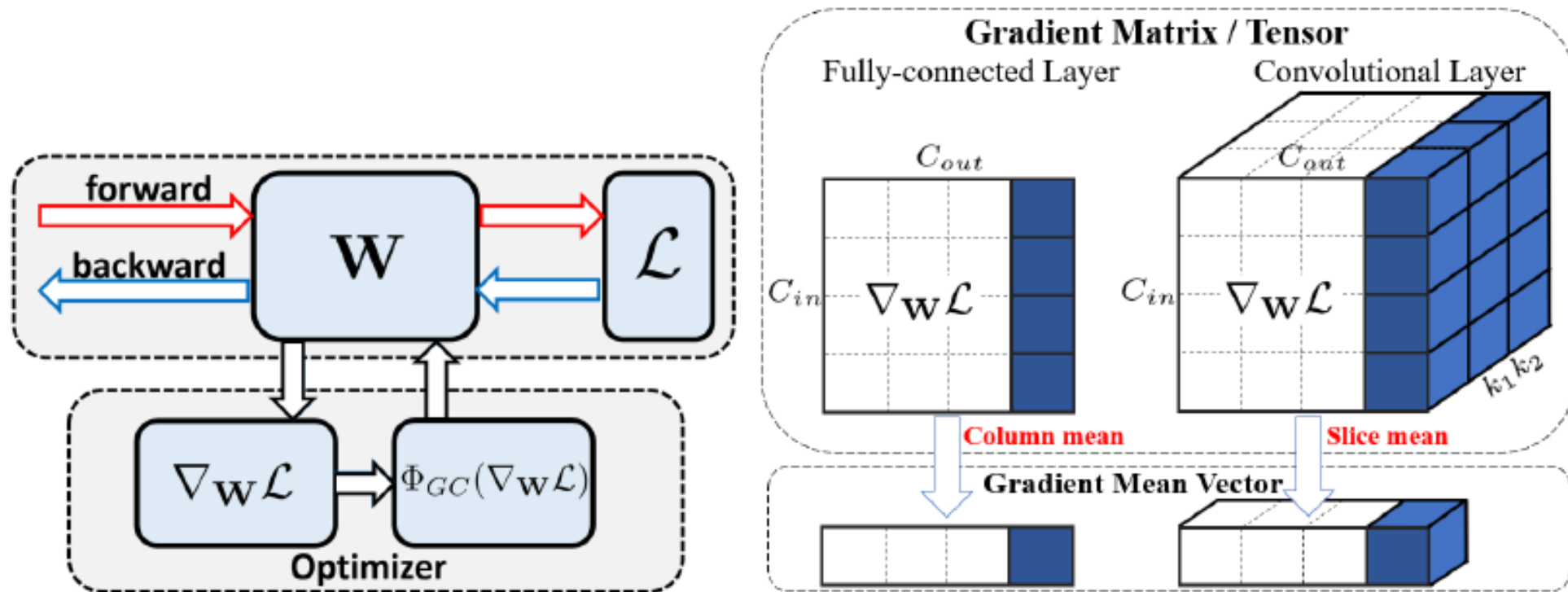
- **Atualização dos pesos**

O **otimizador** (como **SGD**, **Adam** ou **Adagrad**) utiliza o gradiente centralizado para realizar a atualização:

$$W_{t+1} = W_t - \alpha \Phi_{GC}(\nabla_W L)$$



Arquitetura e funcionamento



Treinamento e otimização

Efeitos sobre o treinamento

A aplicação do GC gera diversos efeitos positivos na prática:

- **Convergência mais rápida:** o modelo atinge valores mínimos da função de perda em menos iterações, pois os gradientes tornam-se mais estáveis e previsíveis.
- **Maior estabilidade do aprendizado:** ao eliminar componentes redundantes dos gradientes, reduzem-se as oscilações bruscas e evita-se a divergência durante o treinamento.
- **Melhor equilíbrio entre aprendizado e generalização:** o GC atua como uma forma de regularização implícita, evitando o *overfitting* sem introduzir novos hiperparâmetros.

Em conjunto, esses efeitos tornam o **processo de treinamento mais controlado e eficiente**, especialmente em **redes profundas** ou com **dados complexos**.

Treinamento e otimização

Estratégias de otimização

O GC se **integra aos otimizadores existentes** (*SGD, Adam, Adagrad*, etc.) **sem necessidade de modificar seus parâmetros**.

No entanto, na prática, observam-se alguns benefícios:

- A **taxa de aprendizado** pode ser **ligeiramente aumentada** sem perder estabilidade.
- É possível **alcançar a convergência em menos épocas**, pois o treinamento torna-se mais estável.
- O método apresenta **melhor desempenho com diferentes otimizadores**, mantendo a **mesma configuração**.

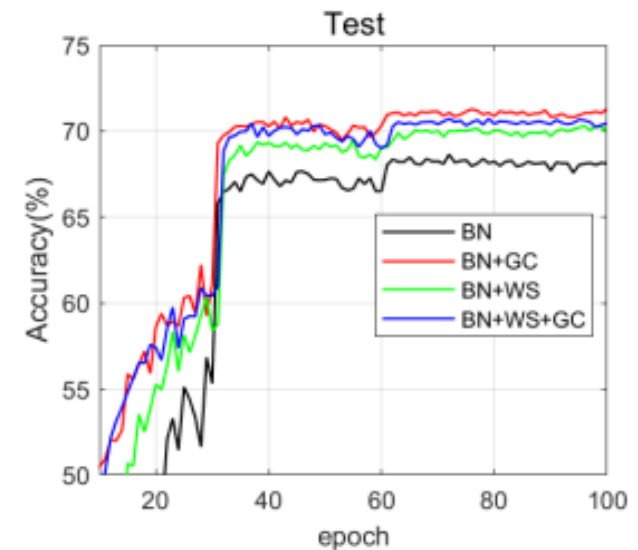
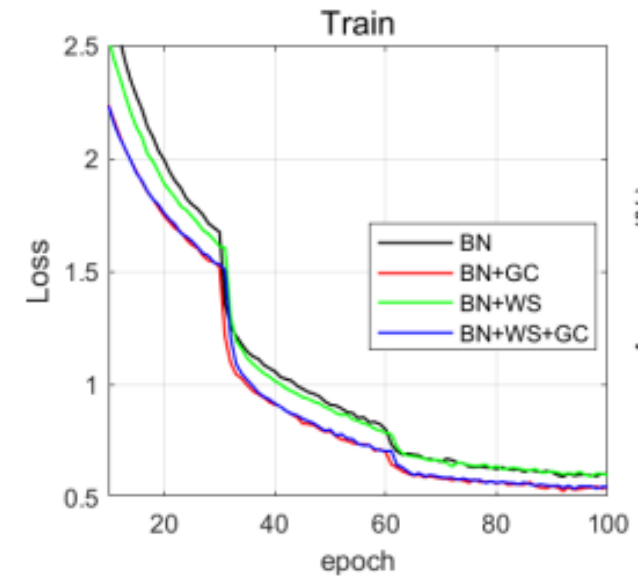
Treinamento e otimização

Dois gráficos comparando diferentes combinações de normalização y GC.

Que demonstra

- As curvas com **GC (vermelho e azul)** descendem y estabilizam antes → treinamento más rápido.
- Em validação, GC antiga **maior precisão final**

GC **acelera o treinamento y melhora a generalização**, incluso combinado com outros métodos como *BatchNorm* y *Weight Standardization*.



Vantagens e desvantagens

Vantagens:

- **Simplicidade de implementação**
O GC pode ser facilmente **incorporado** em qualquer otimizador baseado em **gradientes** (como *SGD* ou *Adam*) com **apenas uma linha de código**.
Não é necessário modificar a arquitetura da rede nem adicionar novos parâmetros.
- **Compatibilidade universal**
Pode ser aplicado em **qualquer tipo de modelo** — redes convolucionais, densas ou de transferência — e até mesmo em **modelos pré-treinados**, ao contrário de outros métodos de normalização que exigem pesos iniciais específicos.
- **Estabilidade do treinamento**
Ao eliminar o componente médio dos gradientes, o GC **reduz a variância e as oscilações** durante a otimização, evitando **explosões ou desaparecimento de gradientes**.

Vantagens e desvantagens

Vantagens:

- **Regularização implícita e melhor generalização**
O GC atua como um **mecanismo natural de regularização**, limitando o espaço de soluções possíveis e evitando o *overfitting* sem necessidade de introduzir hiperparâmetros adicionais.
- **Eficiência computacional**
A operação de centralização tem **custo computacional desprezível e não afeta o tempo total de treinamento**.
Os experimentos mostram **melhorias de desempenho** em múltiplos *datasets* (*CIFAR100*, *ImageNet*, *COCO*, etc.).

Vantagens e desvantagens

- **Desvantagens:**

- **Ganhos moderados**

Embora os resultados sejam consistentes, as melhorias costumam ser pequenas (entre 0,5% e 2% em precisão).

O GC não substitui técnicas de otimização mais complexas — ele as complementa.

- **Dependência do otimizador base**

O impacto positivo do GC pode variar conforme o otimizador utilizado.

Em otimizadores muito adaptativos (como Adam ou RMSProp), o efeito pode ser menos pronunciado.

Vantagens e desvantagens

- **Desvantagens:**

- **Ausência de controle direto**

Por não incluir novos hiperparâmetros, o GC não oferece mecanismos de ajuste fino sobre a magnitude da centralização, o que limita sua personalização.

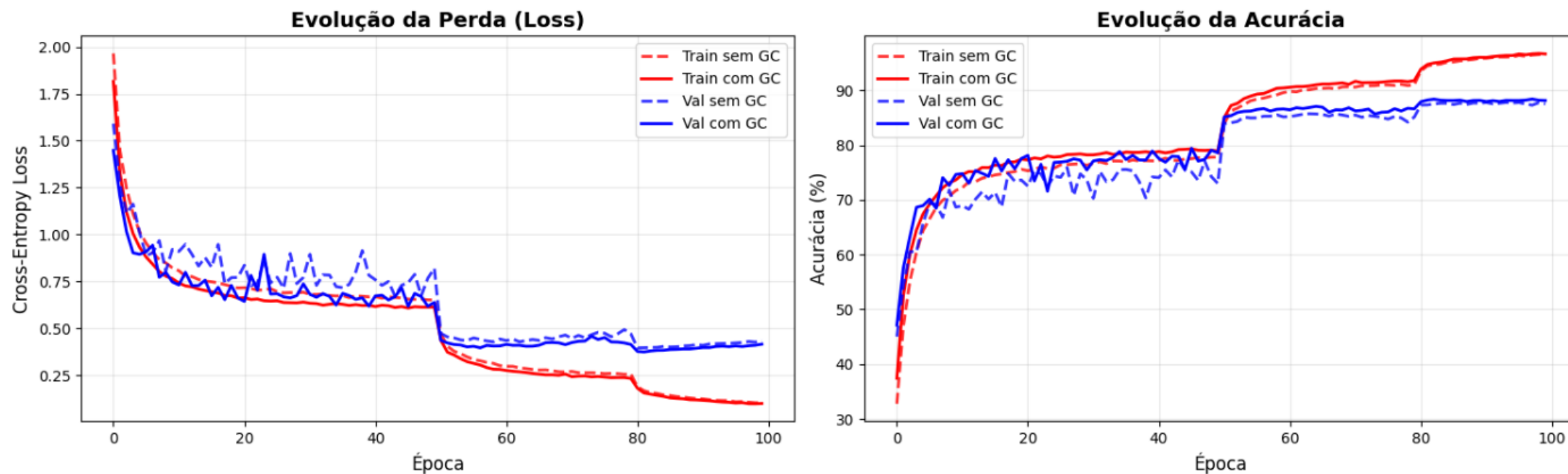
- **Evidência empírica limitada fora da visão computacional**

A maioria das avaliações foi realizada em tarefas de visão computacional (classificação, detecção, segmentação).

Sua eficácia em domínios como processamento de linguagem natural ou séries temporais ainda requer validação adicional.

Exemplo(s) de aplicação

Comparação Gradient Centralization



RESULTADOS FINAIS NO CONJUNTO DE TESTE

SEM GC -> loss: 0.4326 | accuracy: 0.8752 (87.52%)

COM GC -> loss: 0.4158 | accuracy: 0.8813 (88.13%)

Melhoria com GC: +0.61 pontos percentuais

Exemplo(s) de aplicação

Avalia-se o efeito do GC em diferentes arquiteturas (ResNet, VGG, DenseNet, ResNeXt) utilizando o dataset CIFAR-100

Interpretação

- O **GC** melhora a **precisão em todas as redes**, mesmo em arquiteturas muito diferentes.
- O ganho médio é de **1–2%**, o que é significativo no CIFAR-100.

Conclusão

- O **GC** é um método **geral e adaptável**: melhora qualquer arquitetura **sem necessidade de modificações**.

Table 1. Testing accuracies of different DNN models on CIFAR100

Model	R18	R101	X29	V11	D121
w/o GC	76.87±0.26	78.82± 0.42	79.70±0.30	70.94± 0.34	79.31±0.33
w/ GC	78.82±0.31	80.21±0.31	80.53±0.33	71.69±0.37	79.68±0.40

Exemplo(s) de aplicação

Contexto

- O **GC** foi testado com vários algoritmos de otimização: **SGDM, Adam, Adagrad, SGDW e AdamW**.

Conclusão

- O **GC** melhora o desempenho de **todos os otimizadores**, inclusive dos **adaptativos**, confirmando sua **compatibilidade universal**.

Table 2. Testing accuracies of different optimizers on CIFAR100

Algorithm	SGDM	Adam	Adagrad	SGDW	AdamW
w/o GC	78.23±0.42	71.64±0.56	70.34 ±0.31	74.02±0.27	74.12±0.42
w/ GC	79.14±0.33	72.80±0.62	71.58±0.37	76.82±0.29	75.07±0.37

Exemplo(s) de aplicação

Contexto

- Estuda-se se o **GC** depende do **hiperparâmetro de regularização (weight decay)**.

Conclusão

- O **GC** mantém ou melhora a **precisão** em todos os níveis de regularização, demonstrando **robustez em relação ao *weight decay***.

Table 3. Testing accuracies of different weight decay on CIFAR100 with ResNet50.

Weight decay	0	$1e^{-4}$	$2e^{-4}$	$5e^{-4}$	$1e^{-3}$
w/o GC	71.62±0.31	73.91±0.35	75.57±0.33	78.23±0.42	77.43±0.30
w/ GC	72.83±0.29	76.56±0.31	77.62±0.37	79.14±0.33	78.10±0.36

Exemplo(s) de aplicação

Contexto

- Avalia-se se o **GC** depende da **taxa de aprendizado (learning rate)** utilizada durante o treinamento.

Conclusão

- O **GC** funciona bem com **qualquer taxa de aprendizado**, o que **facilita seu uso** sem necessidade de reajustar hiperparâmetros.

Table 4. Testing accuracies of different learning rates on CIFAR100 with ResNet50 for SGDM and Adam.

Algorithm	SGDM	SGDM	SGDM	Adam	Adam	Adam
Learning rate	0.05	0.1	0.2	0.0005	0.001	0.0015
w/o GC	76.81±0.27	78.23±0.42	76.53±0.32	73.88±0.46	71.64±0.56	70.63±0.44
w/ GC	78.12±0.33	79.14±0.33	77.71±0.35	74.32±0.55	72.80±0.62	71.22±0.49

Exemplo(s) de aplicação

Após avaliar o **GC** em tarefas de **classificação geral** (*CIFAR*, *ImageNet*), os autores testam sua eficácia em **tarefas de *fine-tuning***, ou seja, quando um **modelo pré-treinado (ResNet-50)** é ajustado sobre **conjuntos de dados mais específicos e detalhados**:

- FGVC Aircraft ✈️
- Stanford Cars 🚗
- Stanford Dogs 🐕
- CUB-200-2011 (aves) 🦅

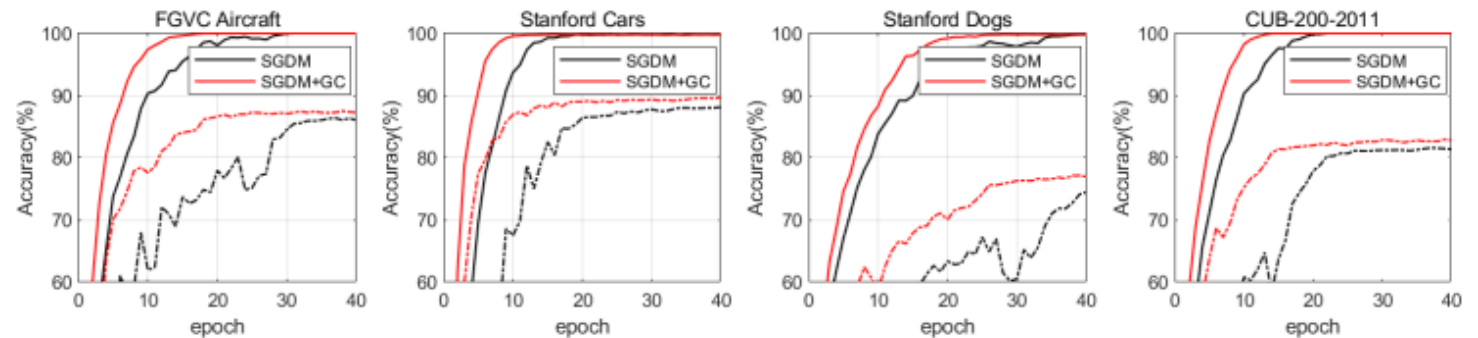


Fig. 7. Training accuracy (solid line) and testing accuracy (dotted line) curves vs. training epoch on four fine-grained image classification datasets.

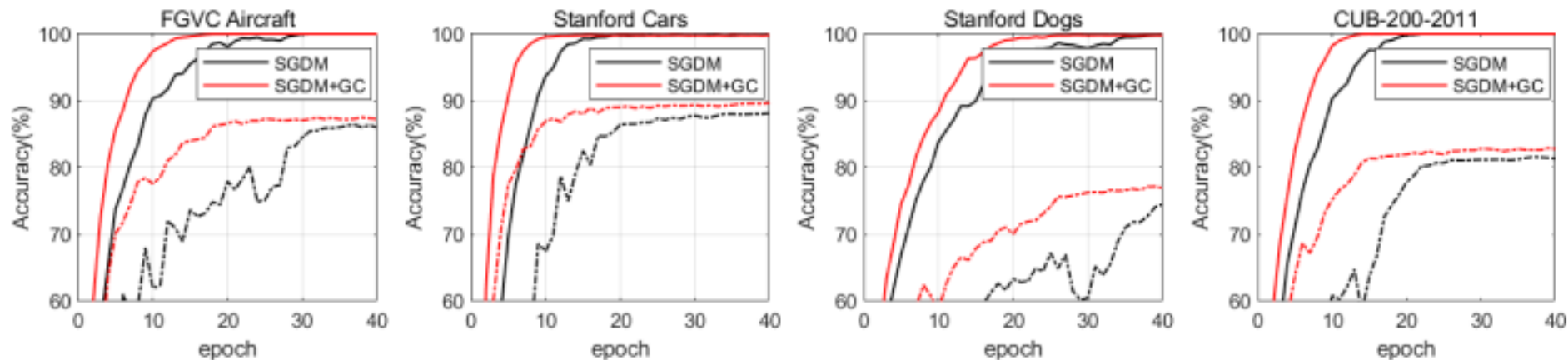
Fig. 7 shows the training and testing accuracies of SGDM and SGDM+GC for the first 40 epochs on the four fine-grained image classification datasets. Table 7 shows the final testing accuracies. We can see that both the training and testing

Exemplo(s) de aplicação

O que mostra

Cada gráfico apresenta a **evolução da precisão (accuracy %)** ao longo das **épocas de treinamento**:

- **Linha contínua:** precisão no treinamento.
- **Linha pontilhada:** precisão no teste (*testing*).
- **Preto:** treinamento padrão com **SGDM**.
- **Vermelho:** treinamento com **SGDM + GC**.



Exemplo(s) de aplicação

Interpretação

Nos quatro *datasets*, as **curvas vermelhas (com GC)** ficam **acima das pretas** desde as primeiras épocas.

→ O **GC** permite **aprender mais rápido** e alcançar **maior precisão final**.

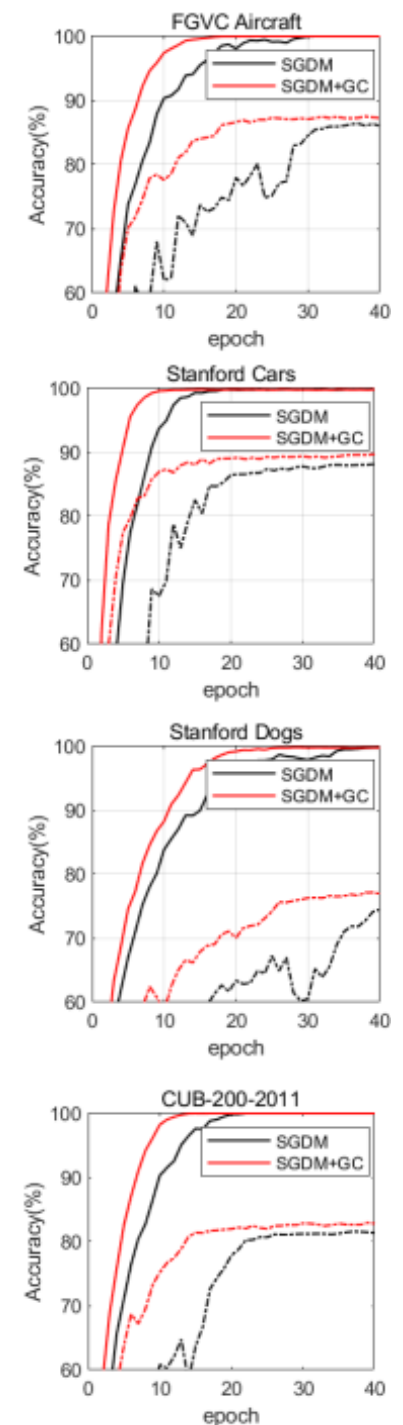
A **linha pontilhada vermelha (teste)** também se estabiliza antes e atinge valores mais altos.

→ O **GC** melhora a **generalização do modelo**, não apenas o aprendizado no treinamento.

A diferença é especialmente perceptível em **Stanford Dogs** e **CUB-200-2011**, onde o aprendizado sem GC é mais irregular.

O que demonstra

- O **GC** acelera a **convergência** no treinamento de modelos pré-treinados.
- Melhora a **precisão em teste** em todas as tarefas de **classificação detalhada**.
- É **robusto** em diferentes domínios de dados (veículos, animais, aves, aviões).
- Reforça o mostrado na **Tabela 7**, onde os ganhos de precisão variam de **+1% a +2%**



Exemplo(s) de aplicação

Utilizam-se **datasets com classes visualmente muito semelhantes** para testar a **capacidade de generalização do GC**.

Contexto

- Resultados do *fine-tuning* com e sem **GC**.

Conclusão

- O **GC** melhora a **precisão entre 1% e 2%**, mesmo em *fine-tuning* com **modelos pré-treinados**, confirmando sua **compatibilidade e efetividade universal**.

Table 6. The statistics of fine-grained datasets used in this paper.

Datasets	#Category	#Training	#Testing
FGVC Aircraft	100	6,667	3,333
Stanford Cars	196	8,144	8,041
Stanford Dogs	120	12,000	8,580
CUB-200-2011	200	5,994	5,794

Table 7. Testing accuracies on the four fine-grained image classification datasets.

Datesets	FGVC Aircraft	Stanford Cars	Stanford Dogs	CUB-200-2011
w/o GC	86.62±0.31	88.66±0.22	76.16±0.25	82.07±0.26
w/ GC	87.77±0.27	90.03±0.26	78.23±0.24	83.40±0.30

Exemplo(s) de aplicação

Contexto

- Avalia-se o **impacto do GC** na **detecção de objetos**, uma tarefa mais complexa do que a classificação.

Conclusão

- O **GC** aumenta a **precisão média (AP)** em **todos os modelos**, inclusive em **arquiteturas muito grandes**, resultando em um **treinamento mais estável em tarefas de detecção**.

Table 8. Detection results on COCO by using Faster-RCNN and FPN with various backbone models.

Method	Backbone	AP	AP _{.5}	AP _{.75}	Backbone	AP	AP _{.5}	AP _{.75}
w/o GC	R50	36.4	58.4	39.1	X101-32x4d	40.1	62.0	43.8
w/ GC	R50	37.0	59.0	40.2	X101-32x4d	40.7	62.7	43.9
w/o GC	R101	38.5	60.3	41.6	X101-64x4d	41.3	63.3	45.2
w/ GC	R101	38.9	60.8	42.2	X101-64x4d	41.6	63.8	45.4

Exemplo(s) de aplicação

Contexto

- Avalia-se o **GC** simultaneamente em **detecção (AP^b)** e **segmentação (AP^m)**.

Conclusão

- O **GC** melhora tanto a **detecção** quanto a **segmentação**, confirmando que seu **efeito positivo se estende além da classificação**.
Em **visão computacional complexa**, pequenas melhorias de AP (~0,5–1,0) são significativas.

Table 9. Detection and segmentation results on COCO by using Mask-RCNN and FPN with various backbone models.

Method	Backbone	AP ^b	AP ^b _{.5}	AP ^b _{.75}	AP ^m	AP ^m _{.5}	AP ^m _{.75}	Backbone	AP ^b	AP ^b _{.5}	AP ^b _{.75}	AP ^m	AP ^m _{.5}	AP ^m _{.75}
w/o GC	R50	37.4	59.0	40.6	34.1	55.5	36.1	R50 (4c1f)	37.5	58.2	41.0	33.9	55.0	36.1
w/ GC	R50	37.9	59.6	41.2	34.7	56.1	37.0	R50 (4c1f)	38.4	59.5	41.8	34.6	55.9	36.7
w/o GC	R101	39.4	60.9	43.3	35.9	57.7	38.4	R101GN	41.1	61.7	44.9	36.9	58.7	39.3
w/ GC	R101	40.0	61.5	43.7	36.2	58.1	38.7	R101GN	41.7	62.3	45.3	37.4	59.3	40.3
w/o GC	X101-32x4d	41.1	62.8	45.0	37.1	59.4	39.8	R50GN+WS	40.0	60.7	43.6	36.1	57.8	38.6
w/ GC	X101-32x4d	41.6	63.1	45.5	37.4	59.8	39.9	R50GN+WS	40.6	61.3	43.9	36.6	58.2	39.1
w/o GC	X101-64x4d	42.1	63.8	46.3	38.0	60.6	40.9							
w/ GC	X101-64x4d	42.8	64.5	46.8	38.4	61.0	41.1							

Comparação com outros algoritmos

- El O método Gradient Centralization (GC) distingue-se de outros algoritmos de otimização e normalização por sua **simplicidade conceitual** e por atuar **diretamente sobre os gradientes**, em vez de agir sobre as **ativações** ou os **pesos** do modelo.

Comparação com outros algoritmos

1. Comparação com técnicas de normalização

Técnica	Atua sobre	Princípio	Limitações
Batch Normalization (BN)	Ativações	Estandariza as saídas de cada camada usando a média e a variância do mini-lote.	Seu desempenho depende do tamanho do lote; pode falhar com lotes pequenos.
Weight Standardization (WS)	Pesos	Normaliza os vetores de pesos para que tenham média zero e variância unitária.	Não é compatível com modelos pré-treinados, pois requer pesos iniciais padronizados.
Gradient Centralization (GC)	Gradientes	Centraliza os gradientes subtraindo sua média, reduzindo a redundância direcional.	Não altera ativações nem pesos, aplica-se apenas na etapa de otimização.

Comparação com outros algoritmos

2. Comparação com algoritmos de otimização

Otimizador	Características	Relação com o GC
SGD / SGDM	Ajusta os pesos na direção do gradiente; simples e amplamente utilizado.	O GC pode ser integrado diretamente, reduzindo oscilações e acelerando a convergência.
Adam	Adapta a taxa de aprendizado para cada parâmetro; rápido, porém menos estável.	O GC melhora sua estabilidade sem afetar sua adaptabilidade.
Adagrad / RMSProp	Ajustam dinamicamente a taxa de aprendizado acumulando gradientes.	O GC suaviza os gradientes, permitindo atualizações mais consistentes.

Comparação com outros algoritmos

3. Situações em que o GC se destaca

- **Treinamentos com lotes pequenos:**
Diferente do *Batch Normalization (BN)*, o GC não depende do tamanho do mini-lote, mantendo a estabilidade mesmo com poucos dados por iteração.
- **Modelos pré-treinados:**
Pode ser aplicado diretamente em tarefas de *fine-tuning*, pois **não exige pesos inicializados de forma normalizada**, ao contrário do *Weight Standardization (WS)*.
- **Redes profundas ou com gradientes instáveis:**
O GC reduz a variabilidade e previne **explosões de gradiente**, favorecendo uma **convergência mais estável**.
- **Cenários com recursos limitados:**
Sua implementação é **extremamente leve**, sem custos adicionais de **memória** ou **tempo de computação**.

Perguntas?

Referências

- Young Hongwei, et al., “Gradient Centralization: A New Optimization Technique for Deep Neural Networks”,
<https://arxiv.org/pdf/2004.01461>.
- Repositório do GitHub: <https://github.com/Yonghongwei/Gradient-Centralization>

Links

- GitHub:
https://github.com/Mish0404/TP558/tree/main/Seminario_Gradient_Centralization
- Quiz: [Gradient Centralization](#)

Obrigado!